SN754410NE H-Bridge Circuit Analysis

and How It Works

PARMCO — ECSE 4235 Final Project | Rev B (opto-isolated enable on GPIO 26) | Macey Glad

1. Design Requirements and How the Circuit Meets Them

Requirement	How the circuit satisfies it
Motor disabled whenever the RP4 GPIO is LOW or floating, including during boot	GPIO 26 can only turn the driver on by supplying several mA to the 4N25 LED. LOW or floating supplies none, so the phototransistor stays off and the 10 kΩ pull-down (R2) holds DRV_EN at 0 V. The hardware enforces this, not the software.
Enable is on/off only (no PWM)	DRV_EN is a static logic level. Speed and direction come only from the step sequence on GPIO 18/23/24/25.
Idle state de-energizes the coils (Feedback hold)	When the motor is idle, firmware drives GPIO 26 LOW. The SN754410NE outputs go high-impedance, so no coil current flows.
VCC1 (logic) and VCC2 (motor) from the in-class DC supply, never the RP4	U2 pin 16 goes to bench +5.0 V, and U2 pin 8 goes to bench +12.0 V. The 4N25 collector (pin 5) also uses bench +5 V. No RP4 power pin connects to U2.
Shared reference	The RP4 GND (Pin 6), bench GND, U2 pins 4/5/12/13 and the 4N25 cathode are tied together so the 3.3 V phase signals have a common reference.

2. Circuit Overview

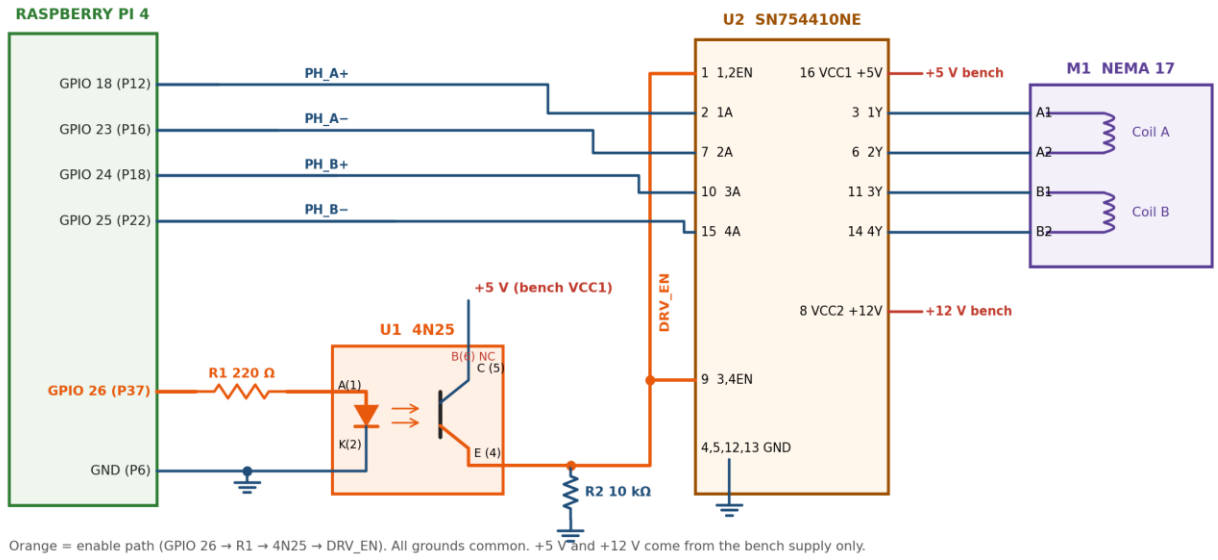


Figure 1 — Motor control circuit. The enable path is shown in orange.

Signal	RP4 pin	Goes to	Purpose
PH_A+	GPIO 18 (Pin 12)	U2 pin 2 (1A)	Drives output 1Y (pin 3) to Coil A1
PH_A−	GPIO 23 (Pin 16)	U2 pin 7 (2A)	Drives output 2Y (pin 6) to Coil A2
PH_B+	GPIO 24 (Pin 18)	U2 pin 10 (3A)	Drives output 3Y (pin 11) to Coil B1
PH_B−	GPIO 25 (Pin 22)	U2 pin 15 (4A)	Drives output 4Y (pin 14) to Coil B2
ENABLE	GPIO 26 (Pin 37)	R1 220 Ω → 4N25 pin 1	Turns the 4N25 LED on and off
DRV_EN	—	4N25 pin 4 → U2 pins 1 & 9, R2 10 kΩ to GND	Master enable for both H-bridges

Signal	RP4 pin	Goes to	Purpose
GND	Pin 6	4N25 pin 2, U2 GND pins, bench GND	Common reference

3. Enable Circuit Analysis (4N25)

3.1 Enabled state: GPIO 26 HIGH

GPIO 26 drives 3.3 V into the 220 Ω resistor and the 4N25 infrared LED. With a typical LED forward voltage of about 1.2 V:

```
I_F = (V_GPIO − V_F) / R1 = (3.3 V − 1.2 V) / 220 Ω ≈ 9.5 mA
```

This is within the Pi's per-pin output rating and well below the 4N25 LED's 60 mA maximum. If the Pi pin sags to about 3.0 V under load, I_F is still about 8 mA, which is enough.

The phototransistor acts as a **high-side switch (emitter follower)**. Its collector is on the bench +5 V and its emitter drives DRV_EN. The 4N25's minimum current transfer ratio (CTR) is 20%, so it can sink at least:

```
I_C(available) ≥ 0.20 × 9.5 mA ≈ 1.9 mA
```

The load only needs current for R2 plus two enable inputs. Each enable input draws only microamps:

```
I_C(needed) ≈ 5 V / 10 kΩ + 2 × I_IH ≈ 0.5 mA + negligible
```

The transistor has about 4× more current than the load needs, so it saturates. DRV_EN is about 5 V − V_CE(sat) ≈ **4.5–4.8 V**, comfortably above the SN754410NE's 2.0 V minimum logic-HIGH threshold (V_IH). Both half-H pairs turn on.

3.2 Disabled state: GPIO 26 LOW

With GPIO 26 at 0 V, no LED current flows. The phototransistor leaks only its dark current (I_CEO, at most about 50 nA), so R2 pulls DRV_EN to:

```
V_EN = I_CEO × R2 ≈ 50 nA × 10 kΩ = 0.5 mV
```

This is far below the 0.8 V maximum logic-LOW threshold (V_IL). Outputs 1Y–4Y go **high-impedance**, so the coils carry no current regardless of the phase inputs.

3.3 Floating and boot state (quiet start-up)

During power-up, reboot, or before the program runs, GPIO 26 is an input. It defaults to an internal pull-down, and the pin may also simply float. A floating or high-impedance pin cannot **source** the milliamps the LED needs to conduct. So the LED stays dark, the transistor stays off, and R2 holds DRV_EN LOW. The motor cannot be enabled by:

- the Pi booting, or the 3.3 V rail ramping up;
- random states on GPIO 18/23/24/25 before software configures them;
- the bench supply being switched on before or after the Pi;
- the Pi being unplugged or the GPIO 26 wire coming loose. A broken enable wire **fails safe** to disabled.

Because R2 sits on the driver side of the optocoupler, the default-off state holds even if the Pi is completely unpowered.

3.4 Why the enable is not PWM'd

- The spec requires on/off control only. Speed is set by the step rate on the phase pins.
- The 4N25 is slow. Turn-on is a few microseconds, but turn-off through a 10 kΩ load is much longer. PWM through it would distort the duty cycle and leave the driver partly on.

- Chopping the enable on an inductive load would cause repeated voltage spikes and heating with no control benefit.

4. H-Bridge Analysis (SN754410NE)

4.1 How the bridge drives a coil

The SN754410NE contains four half-H drivers. Each output (nY) is pulled to VCC2 (+12 V) when its input (nA) is HIGH, and to GND when nA is LOW, but only while its enable pin is HIGH. Two half-H drivers on opposite ends of one coil form a full H-bridge:

- **Coil A** sits between 1Y (pin 3) and 2Y (pin 6), driven by GPIO 18 and GPIO 23.
- **Coil B** sits between 3Y (pin 11) and 4Y (pin 14), driven by GPIO 24 and GPIO 25.

Setting one input HIGH and the other LOW drives current through the coil in one direction. Swapping them reverses the current. Setting both the same puts equal voltage on both ends, so no current flows.

EN (DRV_EN)	1A (GPIO 18)	2A (GPIO 23)	Coil A voltage	Coil A current
LOW	X	X	Outputs high-impedance	Zero (de-energized)
HIGH	1	0	+12 V on A1, 0 V on A2	Forward (A1 → A2)
HIGH	0	1	0 V on A1, +12 V on A2	Reverse (A2 → A1)
HIGH	0	0	Both 0 V	Zero (both ends shorted to GND)
HIGH	1	1	Both +12 V	Zero (both ends shorted to +12 V)

Coil B behaves the same way with GPIO 24 and GPIO 25.

4.2 Step sequence (full-step, two coils on)

Step	GPIO 18 (1A)	GPIO 23 (2A)	GPIO 24 (3A)	GPIO 25 (4A)	Coil A	Coil B
1	1	0	1	0	+	+
2	0	1	1	0	−	+
3	0	1	0	1	−	−
4	1	0	0	1	+	−

Going 1→2→3→4→1 turns the shaft one way, and 4→3→2→1 turns it the other way. Each step is 1.8° on a standard 200-step NEMA 17. Speed is set only by the time between steps:

```
RPM = 60 / (200 × t_step)        e.g. t_step = 5 ms → 60 RPM
```

4.3 Logic compatibility

The SN754410NE inputs treat anything at or above 2.0 V as HIGH and at or below 0.8 V as LOW. The Pi's 3.3 V outputs meet both directly, so the phase lines need no level shifting. Because VCC1 is 5 V from the bench, U2 never drives voltage back onto the Pi pins. Its inputs are high-impedance and draw only microamps.

4.4 Coil current and voltage drops

The SN754410NE uses bipolar transistors, not MOSFETs, so it loses voltage across its switches. Typical drops at about 1 A are 1.4 V on the high side and 1.2 V on the low side (up to 1.8 V each, worst case). The voltage across the coil and the resulting current are:

```
V_coil ≈ 12 V − (1.4 V + 1.2 V) ≈ 9.4 V        I_coil ≈ V_coil / R_coil
```

Measured coil resistance	Approx. coil current at 12 V	Within 1 A/channel limit?
30 Ω	≈ 0.31 A	Yes
12 Ω	≈ 0.78 A	Yes, but runs hot (see 4.5)
4 Ω	≈ 2.3 A	No. The supply current limit must stop this.
2.8 Ω	≈ 3.4 A	No. This motor needs a lower VCC2 or a current-limited supply.

Action: measure your motor's coil resistance (A1–A2 and B1–B2) with a multimeter before connecting 12 V, then set the bench current limit accordingly. Your project documents don't specify the motor's winding resistance, so the rows above are examples.

4.5 Power dissipation and why idle de-energizing matters

```
P_driver ≈ 2 coils × I_coil × (V_drop,high + V_drop,low) ≈ 2 × I × 2.6 V
```

At 0.3 A per coil this is about 1.6 W, which is a lot for a DIP package that dissipates heat mainly through ground pins 4, 5, 12 and 13. In full-step drive, current flows in both coils at every step. A motor left powered at standstill heats the IC and the motor continuously while doing no work. **Setting the enable LOW at idle (Feedback hold) eliminates that idle dissipation.** If more heat-sinking is needed, solder the ground pins to a large copper area.

4.6 Inductive kickback, decoupling, and recommended additions

When an output switches off, the coil's stored energy (½·L·I²) forces a voltage spike. TI's SN754410 datasheet calls for **external fast clamp diodes** on inductive loads, and your parts list does not currently include them. Recommended additions, which are not in the original project documents:

- **8 fast or Schottky diodes** (e.g., 1N5819 or UF4001): one from each output (1Y–4Y) to VCC2, and one from GND to each output.
- **Decoupling capacitors:** 0.1 µF ceramic from VCC1 (pin 16) to GND, and 0.1 µF plus ≥100 µF electrolytic from VCC2 (pin 8) to GND, all placed close to the IC.
- **Optional:** a 10 kΩ pull-down on each phase line (GPIO 18/23/24/25), so the inputs sit at a known level during boot. This isn't required for safety, because DRV_EN already keeps the outputs off.

5. How It Works

5.1 The big picture

The Raspberry Pi is the brain, the SN754410NE is the muscle, and the 4N25 is a safety gate between them. The Pi can only send small 3.3 V signals. The SN754410NE takes those signals and switches the bench supply's 12 V into the motor coils. The 4N25 decides whether the SN754410NE is allowed to do anything at all, and its default answer is **no**.

5.2 Power-up (quiet start-up)

1. The bench supply comes on: +5 V powers the SN754410NE's logic, and +12 V waits at the motor supply pin.
2. The Pi boots. GPIO 26 is an input, so no current reaches the 4N25 LED.
3. With the LED off, the phototransistor is off, and the 10 kΩ resistor holds the driver's enable pins at 0 V.
4. The driver outputs are disconnected (high-impedance), so the motor stays still and the coils stay cold no matter what the phase pins are doing.

5.3 Moving the motor

1. Software sets GPIO 18/23/24/25 as outputs and loads the first step pattern.
2. Software drives GPIO 26 HIGH. About 9.5 mA lights the LED, the phototransistor turns on, and DRV_EN rises to about 4.6 V.
3. Both H-bridges switch on, and the coils are energized according to the step pattern.
4. Software steps through the 4-step table. Each change moves the rotor 1.8°. The order sets the direction, and the delay sets the speed.
5. The AS5600 reports the shaft angle over I²C, and the IR sensor reports speed pulses on GPIO 17, so the software can check that the motor actually moved as commanded.

5.4 Idle and Feedback hold

When the motor reaches its target, software drives GPIO 26 LOW. DRV_EN falls to 0 V, the outputs turn off, and no current flows in the coils, so nothing heats up. The motor is held only by its own detent (magnetic cogging) torque. Software keeps reading the AS5600 encoder while idle. If the shaft drifts outside the allowed tolerance, it re-enables the driver, steps back to the target, and disables again. The exact tolerance and timing should follow your project's Feedback hold requirement, which wasn't in the documents I had.

5.5 What happens if something goes wrong

Event	Result
Program crashes with GPIO 26 HIGH	The motor stays enabled. A software watchdog or exit handler should set GPIO 26 LOW.
Program exits normally, or GPIO cleanup runs	The pin returns to an input, and the motor is disabled by hardware.
Pi loses power or reboots	The LED turns off, and the motor is disabled by hardware.
GPIO 26 wire comes loose	The pin is floating, and the motor is disabled by hardware.
R2 missing or open	Unsafe. DRV_EN floats. Check 4 below catches this.
Bench +5 V off, +12 V on	The driver's logic is unpowered, so outputs are not driven. Always switch the bench supply on with its current limit set.

6. Firmware Rules

- Use BCM numbering: ENABLE_PIN = 26 and PHASE_PINS = [18, 23, 24, 25].

- At startup, set ENABLE LOW **first**, then configure the phase pins, then load a valid step pattern. Only after that set ENABLE HIGH.
- Never PWM GPIO 26. Use only digital writes (HIGH or LOW).
- Set ENABLE LOW whenever motion completes (Feedback hold), and in every exit, exception, and signal handler.
- Never set both inputs of a coil to the same value while trying to move. That stops current in that coil.

7. Hardware Verification Checklist

Confirm each item on the bench before connecting the motor, then again with it connected.

#	Test	How	Expected
1	Supply isolation	Multimeter continuity from U2 pins 8 and 16 to every RP4 header pin	Open circuit
2	Common ground	Resistance from RP4 Pin 6 to bench GND, U2 pins 4/5/12/13, and 4N25 pin 2	≈ 0 Ω
3	Bench settings	Outputs OFF. Set 5.0 V and 12.0 V, with the current limit set from the measured coil resistance	5.0 V / 12.0 V ± 0.2 V
4	Pull-down	Measure 4N25 pin 4 to GND (power off)	≈ 10 kΩ
5	Boot state	Power on and reboot the Pi. Measure U2 pins 1 and 9 throughout	< 0.1 V the whole time; the motor does not move
6	Pi unplugged	Bench on, Pi disconnected. Measure DRV_EN	≈ 0 V
7	Enable ON	Set GPIO 26 HIGH. Measure DRV_EN and the voltage across R1	DRV_EN ≈ 4.5–4.8 V; V_R1 ≈ 2.1 V (≈ 9.5 mA)
8	Enable OFF	Set GPIO 26 LOW	DRV_EN < 0.1 V; outputs float
9	Phase outputs	EN HIGH, no motor. Toggle each GPIO and measure the matching nY pin	HIGH ≈ 10.5 V+, LOW ≈ 0–1.2 V
10	Step test	Run the 4-step sequence slowly, then reverse it	The shaft turns 1.8°/step and reverses cleanly
11	Idle de-energize	Stop motion. Read the bench supply's 12 V current	Drops to ≈ 0 A
12	Feedback hold	Nudge the shaft while idle and watch the encoder and response	Driver re-enables, returns to target, then disables
13	Thermal	Run 2 minutes, then touch-test U2 carefully	Warm, not too hot to touch

8. Assumptions and Sources

The pin assignments, component values (220 Ω and 10 kΩ), supply voltages, and safety and continuity checks come from the PARMCO Hardware Pinout Specification and the Raspberry Pi 4 Stepper Motor Schematic. The enable is moved to GPIO 26 per Rev B. Electrical figures (LED forward voltage, CTR, dark current, SN754410NE thresholds and voltage drops, the clamp-diode requirement) are typical datasheet values. Confirm them against the 4N25 and SN754410NE datasheets used in class. The motor's coil resistance and the detailed Feedback hold requirement were not given in the documents, so they're shown as examples to be measured or confirmed.

Rev D — Checkpoint 2 Additions (matched to the CP2 program)

Sections 1–8 (Rev B) still describe the electrical design correctly. The sections below add the Checkpoint 2 IR speed sensor and describe exactly how the CP2 program (C with the E4235 library, plus the ARM32 `motorDriver.s`) operates the circuit. Section 11.6 lists where the CP2 program behaves differently from the generic description in Rev B Sections 4.2, 5.3, 5.4 and 6.

9. What Changes at Checkpoint 2

CP2 item	Hardware change	How the circuit and program handle it
IR sensor measures RPM	New: FC-51-style IR module U3 on GPIO 17, plus recommended R3 and C3	The module outputs one active-LOW pulse each time a propeller blade passes (2 pulses per revolution). The program times these pulses and computes measured RPM, independently of the commanded speed.
Status display shows IR RPM	None (software)	The status line shows commanded steps/s, commanded RPM, IR RPM and the IR pulse count. It reprints once per second while running and after every speed change.
IR RPM reads 0 when stopped	None (software)	Stopping clears the IR measurement, and the RPM function returns 0 whenever the motor is not running.
Test A: IR RPM vs commanded RPM	None	Section 13 gives reachable test speeds, expected values and the error budget.
CP1 behavior preserved	None	Quiet start, the 4N25 / R2 enable path, keyboard control and one-revolution moves are unchanged. The IR sensor has no connection to DRV_EN.

10. IR Sensor Circuit

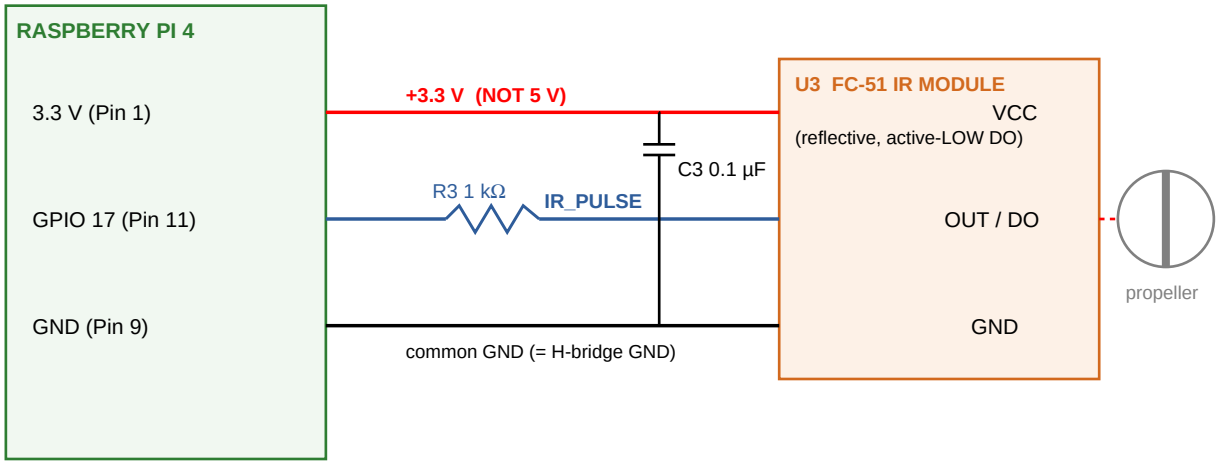


Figure 2 — IR speed sensor wiring. The motor and enable circuit in Figure 1 is unchanged.

10.1 Connections

Signal	RP4 pin	Goes to	Purpose
IR_VCC	3.3 V (Pin 1)	U3 VCC; C3 0.1 µF to GND	Powers the IR LED and comparator at Pi-safe logic level
IR_PULSE	GPIO 17 (Pin 11)	R3 1 kΩ → U3 OUT/DO	Active-LOW pulse per blade (2 per revolution); input only

Signal	RP4 pin	Goes to	Purpose
GND	Pin 9 (any Pi GND)	U3 GND	Same common ground node as Section 2

10.2 Why 3.3 V and not 5 V

The FC-51's DO pin is pulled up to the module's own supply. Powered from 5 V, its HIGH level would be about 5 V, which exceeds the Pi's 3.3 V GPIO limit and can damage GPIO 17. Powered from 3.3 V, DO can never exceed what the Pi accepts, so no level shifter is needed. The program header says the same: the sensor's power and OUT voltage must be verified for the RP4 before connecting.

Powering the sensor from the Pi does **not** break the Rev B supply rule. That rule covers the SN754410NE's VCC1 and VCC2, which still come only from the bench supply. The module draws roughly 20 mA or less from the Pi's 3.3 V rail.

10.3 Supporting parts

- **R3, 1 kΩ in series with GPIO 17:** limits current if GPIO 17 were ever set as an output by mistake. The program configures it only as an input, and R3 does not change the logic level.
- **C3, 0.1 µF across the module supply:** keeps the comparator threshold steady when the IR LED draws current, so it does not create false edges.
- **Pull-up:** the FC-51 board has its own pull-up on DO, and the program does not enable an internal pull-up. If your module has an open-collector output with no pull-up, add one to 3.3 V.
- **Routing:** keep the IR wires away from the coil leads (1Y–4Y), which switch 12 V with sharp edges.

10.4 How the sensor produces a pulse

The module has an IR LED and a photodiode facing the propeller. When a blade passes, IR light reflects back to the detector. An on-board comparator turns this into a clean digital level set by the trim pot: **DO goes LOW while a blade is in view** and HIGH otherwise. The program treats LOW as the active level (`IR_ACTIVE_LEVEL 0`) and counts only the HIGH→LOW transition. This means a blade entering the beam counts once, and leaving the beam does not count again. Check the active level with the sensor in its final mounted position; if your board is inverted, change `IR_ACTIVE_LEVEL` to 1.

Adjust the trim pot so the module's indicator LED lights once per blade when you turn the propeller slowly by hand and stays off with no blade present. Make sure the stand and protractor do not trigger it.

11. How the CP2 Program Drives the Circuit

11.1 Program structure

- **C (main program):** keyboard handling, speed and direction settings, all IR sensor processing, and the status display. All GPIO access in C goes through the E4235 library (`E4235_Init, E4235_Select, E4235_Write, E4235_Read`); no raw register access is used in C.
- **ARM32 assembly (motorDriver.s):** `motor_enable()` drives GPIO 26, `motor_apply_phase()` loads a coil pattern, and `motor_wait_step()` waits one step period and then advances the phase pins one step in the selected direction.
- **One loop, no threads:** a single main loop checks the keyboard, samples the IR sensor, and issues at most one motor step per pass. While running, it polls the keyboard with a 0 ms timeout so stepping is not delayed. While stopped, it waits up to 20 ms per pass.

11.2 Startup sequence (quiet start)

1. `E4235_Init()` opens the GPIO library. If it fails, the program exits before touching any pin.
2. GPIO 26 is set as an output and written LOW **first**. If this fails, the program prints "DO NOT power motor" and exits.

3. GPIO 18, 23, 24 and 25 are set as outputs and written LOW.
4. GPIO 17 is set as an input, and the sensor is read once to learn its resting level.
5. `motor_apply_phase()` loads the first half-step pattern, and `motor_enable(0)` confirms the bridge is off.
6. The terminal switches to single-keystroke mode. The program prints "Startup: bridge DISABLED", the status line (IR RPM 0.00) and the help menu.

This software order backs up, but does not replace, the hardware quiet start in Rev B Section 3.3. Before the program runs, the 4N25 and R2 already hold DRV_EN LOW.

11.3 Keyboard commands

Key	Action	Effect on the circuit
e	Enable bridge	Loads the phase pattern, then GPIO 26 HIGH. Coils are energized and hold position; nothing moves.
d	Disable bridge	Stops any motion, GPIO 26 LOW, coils de-energized, IR measurement cleared.
c / v	Clockwise / counterclockwise	Sets the step direction (c = –1, v = +1, mapped to the observed rotation). Takes effect on the next step.
o	One revolution	Runs exactly 400 half-steps at the current rate, holds the final phase 400 ms, then disables the bridge automatically.
r	Continuous run	Steps continuously at the current rate until x, d or q.
x	Stop	Same as d: stops motion and turns the bridge OFF (coils de-energized). IR RPM shows 0.
f or +	Faster	Raises the step rate by rate/20 (≤ 5%, at least 1 step/s), capped at 3000 steps/s. Prints status.
l or –	Slower	Lowers the step rate by rate/20, floor 30 steps/s. Prints status.
s / h	Status / help	Prints the status line / command list.
q	Quit	Leaves the loop, disables the bridge, restores the terminal and exits.

Speed changes apply to both o and r and can be made while the motor is running. A second o or r while running is refused with "Press x to stop first."

11.4 Step mode and speed

The program runs in **half-step mode** (`STEP_MODE 1`), so one revolution is 400 steps of 0.9° each. Half-stepping alternates between one-coil-on and two-coils-on patterns from motorDriver.s, which gives smoother motion at the low CP2 speeds than the Rev B full-step table. Speed is set only by the time between steps:

```
period_us = round(1,000,000 / rate) RPM_cmd = rate × 60 / 400 = 0.15 × rate
```

Setting	Steps/s	Commanded RPM	Step period
Start-up default	66	9.9	15.15 ms
Minimum (MIN_RATE)	30	4.5	33.3 ms
Maximum (MAX_RATE)	3000	450	0.333 ms

The maximum is a software limit only; the motor will stall well below 450 RPM. Your CP1 stall measurement sets the real upper limit.

11.5 Starting, stopping and one revolution

- **Start (o or r):** load the phase pattern → enable the bridge if it is off → hold 400 ms (`COIL_SETTLE_MS`) so the rotor locks onto the pattern → clear old IR data → begin stepping. Loading the pattern before enabling means the coils are never energized with an undefined pattern.
- **Each step:** `motor_wait_step(period_us, direction, mode)` waits one period and moves the phase pins one half-step. A GPIO error disables the bridge and exits.

- **One revolution (o):** after the 400th step, the program holds the final phase 400 ms, then disables the bridge and prints "One turn command complete: 400 steps issued; bridge OFF." The step count is fixed and does not depend on speed.
- **Stop (x or d):** stepping stops and GPIO 26 goes LOW in the same call, so the coils de-energize immediately and the motor is held only by detent torque.

11.6 Where CP2 differs from the generic Rev B description

Rev B section	Rev B says	CP2 program actually does
4.2 Step sequence	Full step, 4 states, 1.8°/step, RPM = 60/(200 × t_step)	Half step, 8 states, 0.9°/step, RPM = 60/(400 × t_step). At 60 RPM t_step = 2.5 ms, not 5 ms.
4.5 Dissipation	Both coils on at every step	In half step, one coil is on during alternate steps, so average driver dissipation is somewhat lower. Idle de-energizing still applies.
5.3 step 5	AS5600 and IR sensor check motion	CP2 uses only the IR sensor, and only to measure and display RPM. The AS5600 is not read and nothing is corrected automatically.
5.4 Feedback hold	Encoder-based re-enable when the shaft drifts	Not implemented at CP2. After stop or a completed o, the bridge stays OFF until the next command. Encoder hold is a CP3 item.
5.5 Program crash	Watchdog or exit handler should set GPIO 26 LOW	Ctrl+C and SIGTERM set a flag; the loop exits and disables the bridge. An `atexit()` cleanup also disables the bridge and restores the terminal. A hard kill (SIGKILL) or freeze can still leave GPIO 26 HIGH, so keep the bench output switch within reach.
6 Firmware rules	Phase pins configured after ENABLE LOW	Matches: GPIO 26 is written LOW before any other pin is set up (11.2).

12. How the Program Measures IR RPM

12.1 Sampling GPIO 17

The IR sensor is read with `E4235_Read(17)` once per pass of the main loop. While running, each pass contains one motor step, so the sensor is sampled **once per step**. While stopped, it is sampled about every 20 ms so its resting level stays tracked. This is software polling, not a hardware interrupt, so Linux scheduling delays can occasionally cause a missed or late sample. If the read returns anything other than 0 or 1, the program disables the bridge and exits.

Commanded RPM	Steps/s (= samples/s)	Sample spacing	Blade pulse interval
10	≈ 67	15 ms	3.0 s
30	200	5.0 ms	1.0 s
60	400	2.5 ms	0.50 s
90	600	1.67 ms	0.33 s

Because sampling is tied to steps, the number of samples taken while a blade is in the beam is about 1.1 × the blade's width in degrees (400 samples per 360°), at *any* speed. A 10°-wide blade is seen on about 11 samples, which is well above the 2 needed to confirm an edge.

12.2 Debounce and edge detection

- **Level debounce (4 ms):** when a sample differs from the last one, the time is noted. The new level is accepted only if a later sample still shows it at least 4 ms later (`IR_DEBOUNCE_S`). A single noisy sample is ignored.
- **Active edge:** when the accepted level changes to LOW, one blade is recorded at that time. Returning to HIGH records nothing.

- **Minimum spacing (55 ms):** a blade recorded less than 55 ms after the previous one is discarded as chatter (`IR_MIN_PULSE_SPACING_S`). This limits measurement to about 30 / 0.055 ≈ 545 RPM, which is above any speed the motor can reach.
- **Only while running:** blades are recorded only when the motor is running. Hand-turning the propeller while stopped does not create an RPM reading.

12.3 RPM calculation

Each new blade time is subtracted from the previous one to give an interval, timestamped with `CLOCK_MONOTONIC` (not affected by wall-clock changes). The last four intervals (`IR_AVG_INTERVALS 4`, two full revolutions) are stored in a ring buffer:

```
RPM_IR = 60 / (2 × T_avg) T_avg = average of up to 4 most recent intervals
```

Averaging an even number of intervals cancels uneven blade spacing. If one blade is a couple of degrees off 180°, consecutive intervals alternate long/short by about 1%, but every pair of intervals adds up to one exact revolution. With four intervals, the average equals the time for two revolutions divided by four, so sampling error is at most about one step period out of 800 steps (≈ 0.13%) at any speed.

12.4 When IR RPM reads 0 and when it resets

Condition	Result
Motor not running (startup, after x, d, q, or a finished o)	Reads 0.00 immediately. The measurement is also cleared on stop.
Run just started (o or r)	Old data is cleared. Reads 0 until two blades have passed (one interval).
No blade for more than 4.5 s while running	Reads 0. The next blade starts a new measurement (intervals discarded).
Interval longer than 4.5 s	Discarded. This means speeds below 30 / 4.5 ≈ 6.7 RPM read 0, including the 4.5 RPM minimum rate.
Speed changed with f or l while running	Not cleared. The reading moves to the new speed over the next 4 intervals (two revolutions).

12.5 Status display

While running, the status line prints at most once per second (`IR_REPORT_INTERVAL_S`), checked after each step, plus after every f, l, s, x and d. Individual pulses are not printed, because terminal output would delay the next step. Example line:

```
Half step (400 steps/rev); CW (observed); 200 commanded steps/s; 30.00 commanded RPM; IR RPM 29.96; IR pulses 24; bridge ENABLED; RUNNING
```

IR pulses is the number of blades counted since the run started, so at a steady speed it grows by 2 per revolution. It is useful as a quick check against your own count.

13. Test A: IR RPM vs Commanded RPM

13.1 Reaching the test speeds

From the 66 steps/s default, each f adds rate/20 (integer). Pressing only f from startup gives these rates:

Target	f presses from startup	Rate (steps/s)	Commanded RPM	Expected IR pulse interval
30 RPM	25	200	30.00	1.000 s
60 RPM	40	407	61.05	0.491 s
90 RPM	48	597	89.55	0.335 s

*Exactly 60 and 90 RPM are not reachable with the 5% steps. Test A compares IR RPM with the **commanded RPM shown on the same status line**, so use 61.05 and 89.55 as the reference values.*

13.2 Settling before reading

After r, the run starts after the 400 ms hold. A first reading appears after 2 blades, and a full 4-interval average after 5 blades. At 30 RPM that takes about 4.4 s, at 60 RPM about 2.4 s, and at 90 RPM about 1.7 s. Wait at least two revolutions after any speed change before recording a value.

13.3 Error budget

Source	Size	Effect on Test A
Integer step period	≤ 0.5 µs rounding	Negligible.
Loop overhead (sensor read, keyboard check, status printing)	Depends on how motorDriver.s times each step	If motor_wait_step waits a fixed period after each call, overhead adds to every step and the motor runs slightly slower than commanded. The IR reading shows this real speed, which is the point of an independent sensor.
Blade spacing asymmetry	≈ ±1% per interval	Cancelled by the 4-interval (even) average.
Sampling / timestamp resolution	1 step period per edge	≈ 0.13% over the 4-interval average at any speed.
Missed steps	None below stall speed	None if the CP1 stall speed is well above 90 RPM.
30 s blade count × 2 (manual check)	±1 blade = ±2 RPM	±6.7% at 30 RPM. Use the IR pulse count, a 60 s count, or slow-motion video for the independent check.

14. Bluetooth Link

The RP4's built-in Bluetooth radio needs no wiring and uses no GPIO pins, so it does not change the schematic. **This CP2 program does not contain Bluetooth code:** all control is from the keyboard. If Bluetooth messaging is required for your Checkpoint 2 sign-off, it has to be added as a separate program or a later revision. It cannot affect DRV_EN except through software that drives GPIO 26.

15. Updated Pin Map (Rev D)

Signal	RP4 pin	Goes to	Configured by program as
ENABLE	GPIO 26 (Pin 37)	R1 220 Ω → 4N25 pin 1	Output, written LOW first; driven by motor_enable()
PH_A+ / PH_A–	GPIO 18 (Pin 12) / GPIO 23 (Pin 16)	U2 pins 2 / 7	Output, LOW at start; driven by motorDriver.s
PH_B+ / PH_B–	GPIO 24 (Pin 18) / GPIO 25 (Pin 22)	U2 pins 10 / 15	Output, LOW at start; driven by motorDriver.s
IR_PULSE	GPIO 17 (Pin 11)	R3 1 kΩ → U3 DO	**Input only**; read with E4235_Read()
IR_VCC	3.3 V (Pin 1)	U3 VCC, C3	—
GND	Pins 6 and 9	4N25 pin 2, U2 GND pins, bench GND, U3 GND	—
SDA / SCL	GPIO 2 (Pin 3) / GPIO 3 (Pin 5)	AS5600 encoder	Not used by the CP2 program (CP3)

16. How It Works at Checkpoint 2 (end to end)

1. The bench supply and Pi power up. The 4N25 and R2 hold DRV_EN LOW, so the motor is disabled (Rev B 5.2). The FC-51 powers from the Pi's 3.3 V and idles with DO HIGH.
2. The program starts, writes GPIO 26 LOW first, sets the phase pins LOW, sets GPIO 17 as an input, loads the first half-step pattern, and prints "bridge DISABLED" with IR RPM 0.00.

3. The user picks a direction (c or v) and adjusts the rate with f / l. The status line shows the commanded steps/s and RPM.
4. The user presses r. The program loads the pattern, drives GPIO 26 HIGH (about 9.5 mA through the 4N25 LED, DRV_EN ≈ 4.6 V), holds 400 ms, clears the IR data, and starts stepping.
5. Each loop pass: check the keyboard, read GPIO 17, then wait one period and advance one half-step (0.9°) through the SN754410NE.
6. Each time a blade enters the beam, DO goes LOW. After 4 ms of stable LOW, the program records the time. The last 4 intervals are averaged into IR RPM.
7. Once per second, the status line prints commanded RPM next to IR RPM so the two can be compared for Test A.
8. The user presses x (or d). Stepping stops, GPIO 26 goes LOW, DRV_EN drops to 0 V, coil current falls to zero, and IR RPM shows 0.00.
9. With o instead of r, the motor turns exactly 400 half-steps, holds 400 ms, and the bridge switches OFF automatically.
10. On q, Ctrl+C or SIGTERM, the loop exits, the bridge is disabled, the terminal is restored, and the program prints "Program exited with bridge DISABLED."

17. Firmware Rules (as implemented in CP2)

- BCM numbering: enable 26; phases 18, 23, 24, 25; IR 17. GPIO 17 is an input only and never PWM.
- All C-side GPIO access goes through the E4235 library. Enable, phase and step timing stay in motorDriver.s.
- GPIO 26 is written LOW before any other pin is configured, and in every stop, error and exit path.
- The phase pattern is loaded before the bridge is enabled, and a 400 ms settle follows before stepping.
- Stop always disables the bridge (no powered hold at CP2).
- IR processing: 4 ms debounce, count only the active (LOW) edge, ignore spacing under 55 ms, average 4 intervals, 0 after 4.5 s without a pulse or whenever stopped.
- Do not print per pulse; print status at most once per second while running, so stepping is not disturbed.

18. Checkpoint 2 Verification Checklist (adds to Section 7)

#	Test	How	Expected
14	IR supply	Power off: trace U3 VCC to Pi Pin 1 (3.3 V), not to 5 V or the bench	3.3 V source only
15	IR output level	Pi on, motor off: measure GPIO 17 with no blade, then with a blade in the beam	≈ 3.3 V with no blade, ≈ 0 V with a blade; never above 3.3 V
16	Quiet start with sensor	Repeat Section 7 checks 5 and 6 with U3 connected	DRV_EN < 0.1 V; VCC2 current ≈ 0 A
17	Startup screen	Run the program	"bridge DISABLED", STOPPED, IR RPM 0.00
18	Pulse count	Press r at low speed for exactly 1 revolution (or use o) and watch IR pulses	Increases by 2 per revolution
19	Enabled, not running	Press e only, wait 10 s, press s	IR RPM 0.00; coils hold; no false pulses
20	One revolution	Press o at 30 RPM	400 steps, one full turn, then "bridge OFF"; IR RPM 0.00 afterward

#	Test	How	Expected
21	Test A	Run r at 200 / 407 / 597 steps/s; wait 2 revolutions; record IR RPM for 10 s	Within 5% of 30.00 / 61.05 / 89.55 RPM; stable
22	Stop to zero	Press x while running	Bridge DISABLED; 12 V current ≈ 0 A; IR RPM 0.00 immediately
23	Safe exit	Press Ctrl+C while running	Motor stops; "Program exited with bridge DISABLED"; DRV_EN < 0.1 V

19. Assumptions and Sources (Rev D)

Program behavior in Sections 11–17 is taken directly from the CP2 C source: the pins, half-step mode, rate limits, 5% speed steps, 400 ms settle, IR constants (active-LOW, 2 pulses/rev, 4 ms debounce, 55 ms spacing, 4.5 s timeout, 4-interval average, 1 s reporting) and the startup and exit order. The CP2 requirements and Test A come from the ECSE 4235 PARMCO project document. The internal timing of `motor_wait_step()` and the exact half-step patterns are in motorDriver.s, which was not provided, so they are described from the function names and the C comments. The FC-51 behavior (3.3–5 V supply, on-board pull-up, LOW when an object is detected) is typical for this module type; confirm it on your board. R3 and C3 are recommended additions, not parts in the project documents. Bluetooth is not part of this program.